

Building "Too Many Cooks": Research for a Multi-Agent Git Coordination MCP Server

A multi-agent file coordination tool requires integrating MCP protocol mechanics, Dart-to-Node.js compilation, concurrent database access, distributed locking patterns, VSCode visualization, and functional Dart patterns. This research synthesizes implementation details across all six areas to inform a specification optimized for Claude Code implementation using Dart on Node.js via `dart_node`.

MCP SDK architecture enables structured tool coordination

The `@modelcontextprotocol/sdk` (v1.24.3) provides the foundation for building MCP servers. `(npm)` The high-level `(McpServer)` class offers the simplest path to tool registration using **Zod schemas** for input validation:

```
typescript

server.registerTool('acquire_lock', {
  title: 'Acquire File Lock',
  inputSchema: {
    resourcePath: z.string(),
    agentId: z.string(),
    leaseDurationMs: z.number().default(30000)
  },
  outputSchema: { acquired: z.boolean(), leaseId: z.string().optional() }
}, async ({ resourcePath, agentId, leaseDurationMs }) => {
  // Lock acquisition logic
});
```

Transport selection is critical: **stdio transport** suits Claude Desktop and CLI integration (client spawns server as subprocess, communication via stdin/stdout as newline-delimited JSON-RPC).

`(Model Context Protocol)` For multi-client scenarios, **Streamable HTTP transport** supports session management via `(Mcp-Session-Id)` headers and Server-Sent Events for streaming. Session management requires implementing the initialization handshake where clients identify themselves with `(protocolVersion)`, `(capabilities)`, and `(clientInfo)`. `(Model Context Protocol)` `(Modelcontextprotocol)`

Error handling follows two patterns: **protocol errors** use `(McpError)` with standard JSON-RPC codes (`(-32602)` for invalid params), `(DEV Community)` while **tool execution errors** return content with `(isError: true)` to signal failures without breaking the protocol. `(Modelcontextprotocol)` This distinction is essential for graceful degradation.

dart_node compiles Dart to Node.js-compatible JavaScript

`dart_node` enables writing the server in Dart while targeting Node.js. `(dart_node)` It uses **dart2js compiler** (`(dart compile js)`) to produce JavaScript, with **node_preamble** preparing output for Node.js runtime. The compilation flow:

```
bash

dart compile js lib/server.dart -o build/server.js -O4
node build/server.js
```

Interop patterns for wrapping the MCP SDK require understanding three layers:

- 1. **Low-level:** `@JS()` annotations with `package:node_interop` for direct Node.js module access
- 2. **Async bridging:** Convert between Dart `Future` and JS `Promise` using `.toDart` and `.toJS` extensions `Dart`
- 3. **Type definitions:** Use extension types (Dart 3.3+) for type-safe wrappers around JS objects

```
dart

@JS()
external JSPromise fetch(String resource);

// Dart code calling Node.js
Future<void> main() async {
  var response = await fetch('https://api.example.com').toDart;
}
```

Critical limitation: `dart_node` packages remain in beta (v0.8.0-beta). The `node_interop` package provides bindings for `fs`, `path`, `process`, `http` and other core modules, `Pub.dev` but npm packages like `@modelcontextprotocol/sdk` require manual type definitions. The **recommended approach** is creating Dart wrappers around the SDK's TypeScript types using `@JS()` annotations and `require()` imports. `Google Groups` `Pub.dev`

better-sqlite3 with WAL mode handles concurrent lock management

For the lock manager database, **better-sqlite3** with **WAL (Write-Ahead Logging) mode** is optimal. WAL enables **readers to never block writers and vice versa** while maintaining a single-writer constraint:

Operation	Concurrent?
Multiple Readers	✔ Unlimited
Reader + Writer	✔ Non-blocking
Multiple Writers	✘ Serialized

The synchronous API actually **improves concurrency control** by avoiding mutex thrashing—better-sqlite3 benchmarks show **10-15x faster performance** than async alternatives. `github` Enable WAL with:

```
javascript
```

```
db.pragma('journal_mode = WAL');
db.pragma('busy_timeout = 5000');
```

Optimistic locking with version columns implements lease-based locks since SQLite lacks row-level locking:

```
dart
typedef LockRecord = ({
  String resourceId,
  String agentId,
  int acquiredAt,
  int expiresAt,
  int version // For optimistic concurrency
});
```

Use **IMMEDIATE transactions** for write operations to acquire the write lock at transaction start, preventing deadlocks. The pattern: check expiry → validate agent ownership → upsert with version increment → return result.

Alternative consideration: For extreme write contention, **lmdb-js** offers true MVCC with better multi-process support, but as a key-value store lacks SQL query flexibility. Medium

Lease-based advisory locking prevents agent conflicts

Multi-agent coordination requires **advisory locking** (cooperative, all participants check locks voluntarily) rather than mandatory locking (kernel-enforced but deprecated on Linux and unreliable). Git itself uses advisory file-based locks with atomic rename patterns.

Core lock manager design combines lease expiration with heartbeat renewal:

Lock Manager Service			
Lock Table (SQLite)	Lease Manager	Deadlock Prevention	
- resource_id (PK)	- TTL: 30-60s	- Lock ordering	
- agent_id	- Heartbeat	- Timeouts	
- expires_at	- Auto-expire	- Fencing tokens	
- version			

Fencing tokens (monotonically increasing numbers returned with each lock grant) prevent stale writes when GC pauses or network delays cause an agent to believe it holds an expired lock. Include the fencing token in all Git operations.

Lock granularity recommendation: **File-level locking** provides the best balance. Lock individual files instead of entire repositories, avoiding contention on shared metadata. This is achieved by using

being modified, with optional escalation to directory locks for bulk operations. Track file dependencies (imports, includes) to prevent semantic conflicts.

The lock flow follows CrewAI-style hierarchical coordination: Agent requests lock → Lock Manager checks table → Grant with lease ID + fencing token OR Queue agent → On grant, start heartbeat renewal → On completion or timeout, release and notify queued agents.

VSCode extension requires TypeScript but can visualize Dart server state

VSCode extensions must be written in TypeScript/JavaScript for the extension host. Dart cannot be used directly, but the extension can communicate with the Dart MCP server via child process spawn, WebSocket, or Language Server Protocol.

Minimal visualization architecture:

1. **Status Bar Item** — Quick agent activity indicator with Codicon icons

typescript

```
statusBarItem.text = "$(sync~spin) 3 files locked";
statusBarItem.backgroundColor = new vscode.ThemeColor('statusBarItem.warningBackground');
```

2. **File Decorations** — Lock badges (🔒) displayed on files in the explorer via `FileDecorationProvider`

typescript

```
provideFileDecoration(uri: vscode.Uri): vscode.FileDecoration | undefined {
  if (lockedFiles.has(uri.fsPath)) {
    return { badge: '🔒', tooltip: 'Locked by Agent-A', propagate: true };
  }
}
```

3. **Tree View** — Hierarchical display of all locks via `TreeDataProvider`, updated dynamically when lock state changes `visualstudio`

Communication pattern: The extension spawns the MCP server as a child process, then subscribes to lock state changes via stdout JSON messages or a dedicated notification channel. `GitHub` The extension maintains a local cache of lock state and fires `onDidChangeFileDecorations` events when updates arrive.

Functional Dart patterns enable type-safe database access

fpdart (preferred over dartz) provides `Either` and `TaskEither` for type-safe error handling. `Pub.dev` `github` Combined with Dart 3 **records** and **sealed classes**, this creates a robust functional architecture:

Error hierarchy with sealed classes ensures exhaustive pattern matching:

dart

```
sealed class DbError {
    const DbError(this.message);
    final String message;
}
class LockConflict extends DbError { const LockConflict(super.message, this.heldBy); final String heldBy; }
class LockExpired extends DbError { const LockExpired(super.message); }
class ValidationFailed extends DbError { const ValidationFailed(super.message, this.errors); final List<String> errors; }
```

Repository as function collection mimics Haskell type classes:

```
dart

typedef LockRepository = ({
    TaskEither<DbError, LockRecord> Function(String resourceId, String agentId, int ttlMs) acquire,
    TaskEither<DbError, void> Function(String leaseId) release,
    TaskEither<DbError, LockRecord> Function(String leaseId) renew,
    TaskEither<DbError, List<LockRecord>> Function() listActive,
});

LockRepository createLockRepository(Database db) => (
    acquire: (resourceId, agentId, ttlMs) => TaskEither.tryCatch(
        () async => _acquireLock(db, resourceId, agentId, ttlMs),
        (e, _) => LockConflict('Lock held', e.toString()),
    ),
    // ... other operations
);
```

Records for DTOs provide immutable, lightweight data transfer:

```
dart

typedef LockRecord = ({String resourceId, String agentId, int expiresAt, int version, String fencingToken});
typedef AcquireResult = ({bool acquired, LockRecord? lock, String? queuePosition});
```

TaskEither elegantly combines `Future` with `Either` for async database operations:

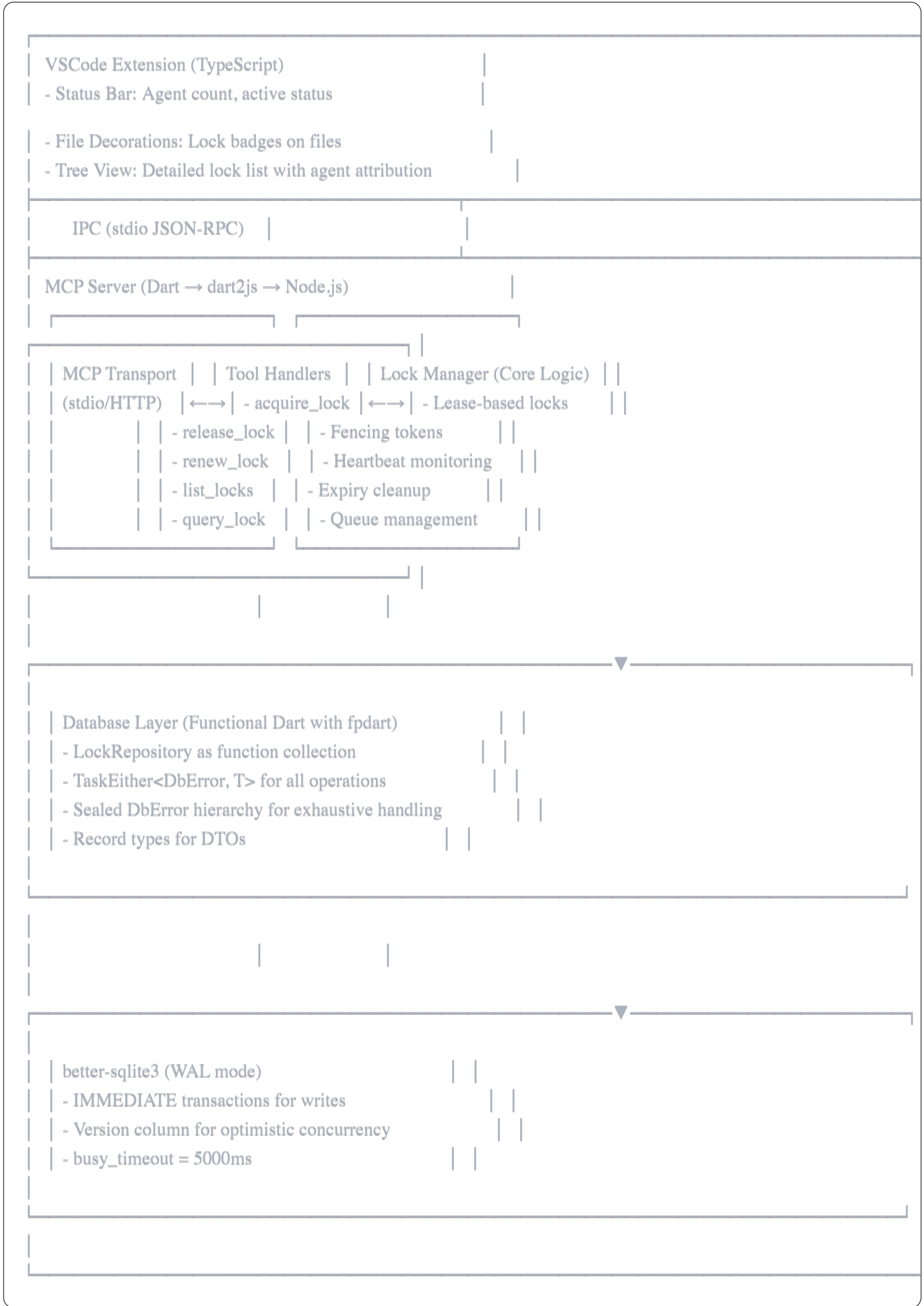
```
dart

TaskEither<DbError, LockRecord> acquireLock(String path, String agent) =>
    validatePath(path)
        .flatMap((_) => checkExistingLock(path))
        .flatMap((existing) => createOrUpdateLock(path, agent, existing));
```

Architectural synthesis for the specification

Architectural synthesis for the specification

The complete architecture integrates these components:



Key specification recommendations:

- **MCP Tools:** `acquire_lock`, `release_lock`, `renew_lock`, `list_locks`, `query_lock`, `get_agent_status`
- **Lease defaults:** 30-60 second TTL with heartbeat renewal at 20-second intervals
- **Database schema:** Single `locks` table with `resource_id` (PK), `agent_id`, `acquired_at`, `expires_at`, `version`, `fencing_token`
- **Error protocol:** Tool errors return `isError: true` in content; protocol errors throw `McpError`
`Modelcontextprotocol`
- **Dart interop:** Wrap better-sqlite3 and MCP SDK with `@JS()` bindings; use `node_interop` for Node.js APIs
- **VSCoDe minimal:** Status bar + file decorations first; tree view as enhancement

Conclusion

This specification foundation enables Claude Code to implement "too many cooks" with confidence. The critical insight is that **advisory locking with lease expiration** provides safe coordination without kernel-level enforcement, `LWN.net` while **better-sqlite3 WAL mode** offers sufficient concurrency for multi-agent scenarios. The functional Dart patterns with fpdart create a type-safe, testable codebase where errors are explicit and exhaustively handled. `Pub.dev` The VSCoDe extension remains a thin TypeScript shell communicating with the Dart server, keeping visualization separate from coordination logic.